

Breaking Down a Problem

Students will be given a real-world problem, such as sorting a large set of emails by priority, and asked to discuss how they would approach solving it manually.

1. Scan email titles:
 - a. Check for important sender e.g. boss or parent
 - b. Look at titles and see if anything looks important
 - c. Look for anything that's obviously advertising/fraud and delete itHandle these first - give priority level 1-10
2. Read each of the remaining emails and give priority level 1-10
3. Sort emails by priority level number.

What are the steps you take when solving a complex problem?

- Highlight obvious wins
- Remove "dead weight"
- Work methodically

How do you break down a problem into smaller, manageable parts?

- Break it down by time - do this, then this, then this
- Or group, these are all similar and can be done at the same time
- Work out if anything is non-essential

Introduction to Computational Thinking

The tutor will provide an overview of computational thinking and its four key aspects:

- **Decomposition** – Breaking down complex problems into smaller sub-problems.
- **Pattern Recognition** – Identifying common structures and similarities.
- **Abstraction** – Removing unnecessary details to focus on essential elements.
- **Algorithmic Thinking** – Designing step-by-step instructions to solve a problem.

Activity: Students will apply computational thinking to a real-world scenario, such as designing an ATM transaction process.

1. User goes to machine
2. Inserts card
3. Enters pin
4. Ask to withdraw money
5. Money received

```

import os
balance = 1000.00

def display_balance():
    return f"Current balance: ${balance:.2f}"

def withdraw():
    global balance

    while True:
        try:
            amount = float(input("Enter the amount to withdraw: "))

            if amount <= 0:
                print("Invalid amount. Please enter a positive
number.")
                continue
            if amount > balance:
                print(f"Insufficient funds. {display_balance()}")
                continue

            balance -= amount
            print(f"You withdrew: ${amount:.2f}")    # <-- prints
withdrawal amount
            print(f"Remaining balance: ${balance:.2f}")
            return amount    # optional, but useful

        except ValueError:
            print("Invalid input. Please enter a numeric value.")

def code():
    correct_pin = "1234"
    attempts = 0
    max_attempts = 3

    while attempts < max_attempts:
        pin = input("Enter your PIN: ")
        if pin == correct_pin:
            print("PIN accepted.")
            print(display_balance())

```

```

        withdraw()
        return
    else:
        attempts += 1
        if attempts < max_attempts:
            print(f"Incorrect PIN. You have {max_attempts -
attempts} attempts left.")
        else:
            print("Account locked due to too many incorrect
attempts.")
            return False

code()

os.system('cls' if os.name == 'nt' else 'clear')

```

Case Study: How do AI-driven recommendation systems use computational thinking to personalise content for users?

Algorithm Design and Problem-Solving Strategies (20 minutes)

Students will explore the principles of algorithm design and how different strategies are used in computing.

Key Discussion Topics:

What makes an algorithm efficient and scalable?

Comparing algorithmic strategies: Brute Force vs. Divide and Conquer vs. Greedy Algorithms.

Introduction to recursion and iterative approaches.

Activity: Students will compare linear search vs. binary search algorithms and determine which is more efficient for different data sets.

Example: How Google Search optimises algorithms to deliver the most relevant search results.

Time and Space Complexity – Understanding Big-O Notation (20 minutes)

Students will be introduced to Big-O notation and why it is important in evaluating algorithm efficiency.

Discussion Topics:

How do we measure the efficiency of an algorithm?

What is the difference between $O(1)$, $O(n)$, $O(\log n)$, and $O(n^2)$ complexity?

Why is time complexity crucial in large-scale computing systems?

Activity: Students will analyse sorting algorithms (Bubble Sort vs. QuickSort vs. Merge Sort) and compare their time complexity.

Case Study: How Facebook and Twitter optimize news feed algorithms using time-efficient data structures.